Trabalho 2

Comparação: Lista Sequencial e Duplamente Encadeada

SCC0502 - Algoritmos e Estruturas de Dados I

ICMC-USP

2026

1 Objetivo

O objetivo desta atividade é colocar em prática os conceitos de **Lista Sequencial** e **Lista Duplamente Encadeada**, analisando empiricamente o impacto das escolhas de representação de memória no desempenho de operações sobre as estruturas.

Ambas as estruturas deverão implementar o mesmo Tipo Abstrato de Dados (TAD): uma **Lista com Cursor**.

2 A Estrutura: Lista com Cursor

Uma Lista com Cursor mantém, além dos elementos armazenados, a referência a uma posição da lista denominada **cursor**. O cursor pode navegar pela lista e serve de âncora para operações locais de inserção, remoção e consulta.

Os índices começam em zero: em uma lista com n elementos, os índices válidos vão de 0 a $n - 1$. O cursor é inválido se, e somente se, a lista estiver vazia. Em uma lista não vazia, ele sempre indica um elemento existente.

O arquivo de cabeçalho `ListaComCursor.h` fornece a interface comum a ser seguida obrigatoriamente por ambas as implementações:

Listing 1: Interface do TAD ListaComCursor (`ListaComCursor.h`)

```c
#ifndef LISTA_COM_CURSOR_H
#define LISTA_COM_CURSOR_H

typedef struct lista_cursor ListaCursor;

// Criacao e Destruicao
ListaCursor* criar_lista(void);
void destruir_lista(ListaCursor* l);

// Navegacao do Cursor
void mover_inicio(ListaCursor* l);
void mover_fim(ListaCursor* l);

// Retorna 1 se avancou; 0 se esta no fim ou a lista esta vazia.
// Em caso de retorno 0, o cursor permanece inalterado.
int avancar_cursor(ListaCursor* l);

// Retorna 1 se retrocedeu; 0 se esta no inicio ou a lista esta vazia.
// Em caso de retorno 0, o cursor permanece inalterado.
int retroceder_cursor(ListaCursor* l);

// Indices comecam em 0. Retorna 1 se o indice for valido;
// caso contrario, retorna 0 e preserva o cursor.
int mover_para_indice(ListaCursor* l, int indice);

// Consulta
// Retorna 1 se obteve o valor; 0 se a lista esta vazia.
// Em caso de retorno 0, *valor_saida permanece inalterado.
int obter_atual(ListaCursor* l, int* valor_saida);

// Modificacao
int inserir_antes(ListaCursor* l, int valor);  // Insere antes do cursor
int inserir_depois(ListaCursor* l, int valor); // Insere depois do cursor
int remover_atual(ListaCursor* l);             // Remove o elemento atual

```

```
36 // Informacao
37 int tamanho_lista(ListaCursor* l);
38
39 #endif
```

2.1 Semântica das Operações

As operações pressupõem que `l` aponta para uma lista criada com sucesso e ainda não destruída. Em `obter_atual`, `valor_saida` deve apontar para um inteiro válido.

Criação e Destruição

- `ListaCursor* criar_lista(void)`: Aloca e retorna uma nova lista vazia e com o cursor em estado inválido. Retorna `NULL` se a alocação falhar.
- `void destruir_lista(ListaCursor* l)`: Libera toda a memória alocada pela lista, incluindo o vetor ou os nós utilizados na implementação.

Navegação do Cursor

- `void mover_inicio(ListaCursor* l)`: Move o cursor para o **primeiro** elemento da lista. Se a lista estiver vazia, não faz nada.
- `void mover_fim(ListaCursor* l)`: Move o cursor para o **último** elemento da lista. Se a lista estiver vazia, não faz nada.
- `int avancar_cursor(ListaCursor* l)`: Avança o cursor uma posição em direção ao fim da lista. Retorna `1` se o cursor avançou com sucesso, ou `0` se o cursor já estava no último elemento (ou a lista está vazia).
- `int retroceder_cursor(ListaCursor* l)`: Retrocede o cursor uma posição em direção ao início da lista. Retorna `1` se o cursor retrocedeu com sucesso, ou `0` se o cursor já estava no primeiro elemento (ou a lista está vazia).
- `int mover_para_indice(ListaCursor* l, int indice)`: Posiciona o cursor no elemento referente ao índice. Retorna `1` se o índice é válido, ou `0` caso contrário (nesse caso, o cursor não é alterado).

Se `avancar_cursor` ou `retroceder_cursor` retornar `0`, o cursor permanece inalterado.

Consulta

- `int obter_atual(ListaCursor* l, int* valor_saida)`:
  Armazena em `*valor_saida` o valor do elemento apontado pelo cursor. Retorna `1` se a operação teve sucesso, ou `0` se a lista está vazia (nesse caso, `*valor_saida` não é modificado).

Modificação

- `int inserir_antes(ListaCursor* l, int valor)`: Insere um novo elemento com o valor dado imediatamente antes da posição do cursor. O cursor permanece apontando para o mesmo elemento de antes da inserção. Se a lista estiver vazia, o novo elemento é inserido e o cursor é posicionado nele. Retorna `1` em caso de sucesso, ou `0` em caso de falha de alocação.
- `int inserir_depois(ListaCursor* l, int valor)`: Insere um novo elemento com o valor dado imediatamente depois da posição do cursor. O cursor permanece apontando para o mesmo elemento de antes da inserção. Se a lista estiver vazia, o novo elemento é inserido e o cursor é posicionado no novo item. Retorna `1` em caso de sucesso, ou `0` em caso de falha de alocação.
- `int remover_atual(ListaCursor* l)`: Remove o elemento atualmente apontado pelo cursor. Após a remoção, o cursor passa a indicar o sucessor do elemento removido. Se não houver sucessor, passa a indicar o predecessor. Se a lista ficar vazia, o cursor fica em estado inválido. Retorna `1` em caso de sucesso, ou `0` se a lista já estava vazia.

Quando uma inserção falhar, o conteúdo da lista e o cursor devem permanecer inalterados.

Informação

- `int tamanho_lista(ListaCursor* l)`: Retorna o número de elementos atualmente armazenados na lista.

3 Implementações Requeridas

Cada grupo deverá implementar os seguintes arquivos:

1. `lista_sequencial.c`: Implementação utilizando alocação sequencial em vetor dinâmico. O cursor é representado por um índice numérico referente à posição atual no vetor.
2. `lista_encadeada.c`: Implementação utilizando nós duplamente encadeados com ponteiros para o nó anterior e o próximo. O cursor é representado por um ponteiro para o nó ativo.
3. `main.c`: Programa principal que implementa o interpretador de comandos descrito adiante, utilizando funções da interface `ListaComCursor.h`.
4. **Código auxiliar de experimentos**: Programas para geração de casos de teste, coleta de tempos e produção dos dados experimentais. A organização e implementação desses arquivos ficam a critério de cada grupo.

As estruturas internas devem ser definidas nos respectivos arquivos `.c`, preservando o encapsulamento do TAD. Para a correção no Run Codes, cada executável deve ser compilado com `main.c` e apenas uma das duas implementações. O programa principal e os experimentos devem acessar a lista exclusivamente pelas funções da interface.

4 Experimentos e Coleta de Dados

Meça o tempo de execução dos casos de teste a seguir para listas de tamanho 1 000, 50 000, 100 000, 500 000 e 1 000 000. Para cada combinação de implementação, tamanho inicial e caso de teste, calcule a média do tempo total do bloco de operações em pelo menos 10 execuções independentes. Antes de cada execução de cada caso, crie uma nova lista com o tamanho inicial especificado e o mesmo conteúdo para ambas as implementações. A medição deve incluir apenas o bloco de operações do caso: exclua a inicialização, posicionamento inicial do cursor, a geração de valores e índices aleatórios, a leitura e escrita de arquivos, a impressão e a destruição da lista. Realocações de memória provocadas pelas inserções medidas fazem parte do tempo do teste.

1. **Inserções no Início (`inserir_antes`):** Realizar 100 inserções no início da lista. Antes de **cada inserção**, chamar `mover_inicio` e, em seguida, `inserir_antes`. Esse reposicionamento é necessário porque a inserção preserva o elemento indicado pelo cursor. Incluir ambas as chamadas na medição.
2. **Acesso por Índice (`mover_para_indice`):** Realizar 100 acessos a posições aleatórias da lista utilizando `mover_para_indice` e `obter_atual`. Gerar previamente os índices, com distribuição uniforme entre 0 e $n - 1$, permitindo repetições. Usar a mesma sequência de índices nas duas implementações em cada repetição.
3. **Remoções no Início (`remover_atual`):** Posicionar o cursor no início com a função `mover_inicio` e realizar 100 remoções consecutivas através de `remover_atual`, sem reposicionar o cursor entre chamadas.
4. **Remoções no Meio (`remover_atual`):** Com a lista de tamanho inicial n, posicionar o cursor no índice $\lfloor n/2 \rfloor$ usando `mover_para_indice`. Em seguida, realizar 100 chamadas consecutivas a `remover_atual`, sem reposicionar o cursor entre elas.
5. **Percurso Completo (`avancar_cursor`):** Posicionar o cursor no início e percorrer toda a lista do primeiro ao último elemento chamando `avancar_cursor` e `obter_atual`.

Lembre-se de utilizar o TAD para realizar cada operação.

Use o mesmo ambiente e as mesmas opções de compilação para ambas as implementações. Registre a função utilizada para medir o tempo e sua unidade. Caso os tempos sejam próximos da resolução do temporizador, repita o bloco de operações em listas

reinicializadas fora dos intervalos medidos, acumulando apenas os tempos dos blocos e informando o procedimento no relatório.

Documente a política de capacidade e realocação do vetor dinâmico e a estratégia de navegação usada no acesso por índice da lista encadeada. Verifique o sucesso das operações. Execuções com falha de alocação não devem compor a média. Se algum tamanho não puder ser executado por limitação de recursos, registre e justifique essa limitação no relatório.

5 Relatório

- **Introdução:** Breve introdução da estrutura de lista com cursor e implementações utilizadas (sequencial e duplamente encadeada).
- **Metodologia:** Ambiente de execução, compilador e opções de compilação, medição do tempo, número de repetições, geração dos dados e decisões de implementação que afetam o desempenho.
- **Resultados:** Tabelas ou gráficos comparando resultados das duas implementações para cada tamanho de entrada e caso de teste.
- **Discussão:** Discussão sobre as complexidades teóricas esperadas (**não é necessário derivá-las formalmente**) e resultados obtidos. Relacione o custo de cada operação ao tempo total do bloco medido e discuta as vantagens e desvantagens observadas nas duas implementações.

6 Submissão

O relatório em formato PDF e todos os arquivos necessários para compilar e executar o trabalho (`main.c`, cabeçalho, `Makefile`, as duas implementações e códigos auxiliares de teste e coleta de dados) devem ser submetidos pelo **e-disciplinas**.

Adicionalmente, o código deverá ser submetido **separadamente** no run.codes para correção automática. Serão criados **dois exercícios** distintos:

1. **Trabalho 2.1 — Lista Sequencial**: submissão da implementação `lista_sequencial.c`.
2. **Trabalho 2.2 — Lista Encadeada**: submissão da implementação `lista_encadeada.c`.

6.1 Formato da Submissão (run.codes)

Para cada exercício, o grupo deverá enviar um arquivo **.zip** contendo:

- `Makefile` (fornecido)

- `ListaComCursor.h` (fornecido)
- `main.c`
- O arquivo `.c` da implementação correspondente ao exercício (`lista_sequencial.c` ou `lista_encadeada.c`)

Atenção: o arquivo `main.c` deve ser o mesmo para ambos os exercícios (Lista Sequencial e Lista Encadeada).

7 Implementação de `main.c`

O `main.c` é o arquivo responsável por interpretar os comandos para avaliação do run.codes. Ele deve ler opções e parâmetros, executar as operações correspondentes por meio das funções de `ListaComCursor.h`, e imprimir os resultados.

Detalhes do programa

O programa deve:

1. Criar uma lista vazia e entrar em loop lendo opções e parâmetros da entrada padrão, sem imprimir menu. Se a criação falhar, encerrar o programa. Encerrar também ao encontrar o fim da entrada (EOF).
2. Executar a operação correspondente à opção lida:
    - **1 x** Inserir x antes do cursor. Não imprime nada.
    - **2 x** Inserir x depois do cursor. Não imprime nada.
    - **3** Remover elemento atual. Não imprime nada.
    - **4** Consultar elemento atual. Se sucesso, imprimir o valor; senão imprimir `ERRO`.
    - **5** Mover cursor para o início. Não imprime nada.
    - **6** Mover cursor para o fim. Não imprime nada.
    - **7** Avançar cursor. Imprimir `1` se avançou, `0` caso contrário.
    - **8** Retroceder cursor. Imprimir `1` se retrocedeu, `0` caso contrário.
    - **9 p** Mover cursor para a posição p. Imprimir `1` se válido, `0` caso contrário.
    - **10** Imprimir o tamanho atual da lista.
    - **0** Destruir a lista e encerrar o programa.

   Ao encerrar por fim da entrada, a lista também deve ser destruída.

Formato de Entrada

Cada linha contém uma opção e, quando necessário, parâmetros inteiros adicionais, conforme descrito acima.

Formato de Saída

- Imprimir apenas o que for solicitado em cada operação, uma linha por resultado.
- Para consulta em lista vazia (opção 4), imprimir exatamente `ERRO`.

Exemplo

As colunas abaixo são blocos independentes: cada linha da saída corresponde a um comando que produz resultado.

Entrada	**Saída**
1 10	10
1 20	3
1 30	1
4	30
10	1
5	10
7	0
4	
7	
4	
7	
0	

8 Critérios de Avaliação

- Correto uso do TAD e fidelidade à interface `ListaComCursor.h`.
- Boas práticas de programação (organização do código, legibilidade, documentação, ausência de vazamentos de memória).
- Relatório (gráficos/tabelas e discussão dos resultados).

9 Observações Importantes

- O trabalho poderá ser realizado em grupos de até 5 alunos.
- **Prazo para entrega: 16/10/2026 às 23h59.**
- A implementação deve ser feita exclusivamente em linguagem C padrão, sem o uso de bibliotecas de estruturas de dados externas.